SvelteKit Mega App Shell – High-Level Plan

Overview

This document outlines a high-level implementation plan for a SvelteKit-based "mega app" shell that hosts multiple mini-apps, with a UX similar to super-apps like Momo, Grab, ZaloPay, or WeChat.

The focus is on a server-side rendered (SSR) app with client-side interactivity (CSR), using Svelte 5 runes and shadcn-svelte for a sleek, modern UI.

Core Goals

- Provide an app shell with a persistent layout, top app bar, bottom navigation, and central mini-app gallery.
- Support SSR for initial loads and SEO, with hydration enabling rich client-side navigation and stateful mini-apps.
- Model mini-apps as separately-routed modules within the SvelteKit app (hosted apps, not remote), with clear contracts and isolation.
- Use shadcn-svelte components (Sidebar, Tabs, Card, Navigation Menu, Dialog, Sheet, etc.) to achieve a polished mobile-like UI.

Technology Stack

- **Framework**: SvelteKit 2+ with Svelte 5 runes, using the standard project layout generated by `npx sv create`.
- **Language**: TypeScript for type-safe components and mini-app contracts.
- **Styling**: Tailwind CSS with shadcn-svelte's design tokens and theming system.
- **UI components**: shadcn-svelte components (Button, Card, Tabs, Sidebar, Navigation Menu, Sheet, Dialog, Avatar, Badge, Progress,

etc.).
- **Routing**: SvelteKit file-based routing for shell and mini-apps; nested layouts for app shell and mini-app containers.
- **State management**: Svelte runes ($state, $derived, $effect) for local and shared reactive state.

High-Level Architecture

App shell layout hierarchy

- Use a root SvelteKit layout (src/routes/+layout.svelte) for global theming, app chrome (top bar, bottom nav), and a responsive container that mimics a mobile device frame on desktop.
- Define a nested src/routes/(shell)/+layout.svelte that wraps all mini-app routes and provides the shell: header, search/command bar, main area (either mini-app gallery or active mini-app), and a bottom tab bar.
- Use SvelteKit layout data (+layout.ts) to load the list of available mini-apps (from a static registry or an API) and pass it via props.

Mini-app routing model

- Treat each mini-app as a route group under /apps/[slug], e.g. /apps/expense-tracker, /apps/todo, /apps/habit-tracker.
- Use SvelteKit dynamic routes (src/routes/(shell)/apps/[slug]/+page.svelte) plus optional nested routes for multi-screen mini-apps.
- Hydrate mini-app pages on the client so navigation between app shell and mini-app feels instant; use SvelteKit enhanced navigation.

Mini-app metadata and registry

Create a central registry describing each mini-app:

- id, slug, name.
- category (Finance, Productivity, Utilities, Labs, etc.).
- icon (Tailwind-based icon component or SVG path).
- accentColor / theme token (e.g., from-violet-500 to-fuchsia-500).
- shortDescription and optional badge (e.g., "New", "Beta").
- entryPath (e.g., /apps/expense-tracker).

This registry can live as a typed module (e.g., `src/lib/mini-apps/registry.ts`) and be imported in layout load functions.

UX and Visual Design

Shell visual language

- Use a mobile-first layout with a centered card-like phone frame on desktop, edge-to-edge layout on mobile.
- Persistent top app bar with brand logo, user avatar (shadcn Avatar), and a command icon opening a command palette (Command component).
- Bottom navigation (Tabs or custom nav) with primary sections: "Home" (mini-app gallery), "Discover", and "Profile".
- Use shadcn Card, Badge, and Avatar to represent mini-app tiles: card with gradient background, icon, name, description, and status badge (e.g., "Lab", "Popular").

Mini-app gallery

- Grid of mini-app cards, 3–4 per row on mobile, 4–6 per row on desktop.
- Each card supports quick actions via long press or context menu (Context Menu component) to pin/unpin, view details, or open in full screen.
- Cards may show minimal live data (e.g., current balance, tasks remaining) using `$state` and `$derived` for derived summaries.

Navigation patterns

- Tap/click on a mini-app tile to navigate to `/apps/[slug]` inside the shell layout.
- Use shadcn Sheet or Drawer to slide mini-apps in from the right on larger screens to preserve shell context, or full-screen on mobile.
- Provide a universal search/command palette (Command) to quickly open mini-apps or deep links.

Example Mini-Apps to Showcase

1. Expense Tracker mini-app

- Simple overview of daily and monthly spend with a chart preview (static for MVP, later backed by real data).
- Use Card, Progress, and Badge to display total spent, budget, and percentage.
- Actions: quick-add expense (Dialog or Drawer with Input, Select, and Button), filter by category.

2. Todo / Task mini-app

- Minimal todo list with add/complete/archive capabilities using Svelte `$state` for local state.
- Use shadcn Input, Button, Checkbox, and Badge to show tasks and status.
- Provide filter tabs (Tabs) for "All", "Active", "Completed".

3. Habit Tracker mini-app

- Daily habits overview (e.g., "Drink water", "Walk 5k steps") in a grid of cards.
- Use shadcn Card, Progress, and Badge to show streaks and completion.
- Optionally use Calendar or Range Calendar for habit history.

4. Labs / Ideas mini-app

- A mini-app that lists experimental features or prototype ideas, itself acting as a meta-lab.
- Uses shadcn Data Table or Item components to display experiments with status, tags, and links.

SSR, CSR, and Hydration Strategy

- Enable SSR at the SvelteKit root for initial HTML of shell and mini-app gallery.
- Use SvelteKit's default `csr` and `trailingSlash` behavior, with progressive enhancement on the client.

- Ensure interactive components (Drawer, Dialog, Tabs, Command, etc.) rely on client-side hydration; they can be rendered on the server but only become interactive after hydration.
- Use `$state` in root layouts for shell-level client state such as active tab, pinned apps, and theme (light/dark), with persistence via localStorage or cookies.

Data and State Boundaries

Shell-level shared state

- Current user info (name, avatar, plan), theme preference (light/dark/system), pinned apps, and recent apps.
- Stored in a Svelte runes-based store module (`.svelte.ts`) so that multiple components can subscribe to shared reactive state.

Mini-app local state

- Each mini-app manages its own state (tasks, expenses, habits) using `$state` runes within its routes/components.
- For persisted state, use localStorage or an API; the high-level plan abstracts data access behind small service modules to keep components clean.

Contracts between shell and mini-apps

- Shell reads mini-app registry metadata and navigates to entry routes.
- Optional: provide a small mini-app runtime context via Svelte context or a `MiniAppLayout` component to expose shell APIs like "show toast", "open modal", or "open command palette".

Component and File Structure

Suggested project structure

- `src/routes/+layout.svelte` – root layout, theme provider, base html/body wrappers.
- `src/routes/(shell)/+layout.svelte` – app shell layout, top bar, bottom nav, mini-app gallery wrapper.
- `src/routes/(shell)/+layout.ts` – load mini-app registry for shell.

- `src/routes/(shell)/+page.svelte` – default "Home" page showing mini-app gallery.
- `src/routes/(shell)/apps/[slug]/+page.svelte` – mini-app container page; uses `slug` to render correct mini-app component.
- `src/lib/mini-apps/registry.ts` – central registry with typed mini-app metadata.
- `src/lib/mini-apps/expense-tracker/*` – components and helpers for expense tracker.
- `src/lib/mini-apps/todo/*` – components and helpers for todo app.
- `src/lib/mini-apps/habits/*` – components and helpers for habit tracker.
- `src/lib/components/ui/*` – shadcn-svelte components installed via the CLI, following their default structure.

Using shadcn-svelte Effectively

- Install shadcn-svelte in SvelteKit using the official installation guide, then use the CLI to add components as needed (Button, Card, Tabs, Sidebar, Command, Dialog, Drawer, Sheet, etc.).
- Configure theming via CSS variables so the app shell can support light/dark mode and per-mini-app accent gradients.
- Stick to the documented component variants and accessibility patterns (e.g., using proper roles, labels, keyboard navigation) to match the quality of Momo/Grab-like apps.

Performance and Progressive Enhancement

- Keep shell layout minimal and static where possible; heavy logic and data fetching should be in mini-app routes.
- Use SvelteKit data loading (`load` functions) with `fetch` on the server where possible to reduce client bundle size.
- Code-split mini-apps by leveraging SvelteKit's route-based code splitting; mini-app bundles only load when the corresponding route is visited.
- Defer non-critical mini-apps and labs from the initial gallery by using skeleton loaders (Skeleton component) and scroll-based lazy loading.

Security and Isolation Considerations

- Since mini-apps live in the same SvelteKit codebase, ensure clear boundaries: no cross-mini-app imports except through shared primitives and shell APIs.
- For future multi-tenant or third-party hosted apps, the same shell UX can be extended by loading remote mini-apps into iframes or through module federation; this plan currently targets first-party hosted apps.

Phased Implementation Plan

Phase 1 – Foundation

- Initialize SvelteKit project with TypeScript and Tailwind.
- Install and configure shadcn-svelte and base components.
- Implement root and shell layouts with app chrome, bottom nav, and static mini-app gallery using dummy data.

Phase 2 – Mini-App Scaffolding

- Create mini-app registry and types.
- Implement dynamic mini-app route (`/apps/[slug]`) and shell navigation.
- Build MVP versions of Expense Tracker, Todo, and Habit Tracker using shadcn-svelte components.

Phase 3 – UX Polish and State

- Add pinned apps, recent apps, and basic persistence.
- Introduce command palette, context menus, and sheets/drawers for interactions.
- Refine mobile and desktop responsiveness and animations.

Phase 4 – Data, Performance, and Hardening

- Add real data layers (localStorage or API) for mini-apps.
- Optimize SvelteKit data loading, code-splitting, and lazy loading.
- Audit accessibility, keyboard navigation, and dark mode.

This plan is designed to be implemented incrementally: once approved, the next steps are to define concrete SvelteKit route files, shell layout components, and a minimal set of shadcn-svelte-based mini-app UIs to serve as a reference implementation.